



UD3

INTRODUCCIÓN A JAVASCRIPT

3.3 FUNCIONES

MP_0373

Lenguaje de marcas y
Sistemas de gestión de
la información

Introducción

Una función es un bloque de código que realiza una determinada tarea. A base de esos pequeños bloques de código es como se construyen los programas (dividen un problema grande en otros más pequeños).

Por ejemplo, una función puede revisar si una cuenta bancaria es correcta (es un cálculo matemático simple) y es muy posible que esté dentro del código de la página web de un banco.

```
function nombreDeLaFuncion () {  
    // Contenido de la función  
}
```

Las funciones se declaran en JavaScript utilizando la palabra reservada `function` seguida del nombre que se desea dar a la función y entre llaves `{}` ese trozo de código que debe ejecutar la función.

Una vez declarada la función, se puede llamar a su ejecución utilizando el nombre de la función seguido por `()`.

```
console.log("pre function");  
function saludaPorConsola() {  
    console.log("Hola, soy JavaScript");  
}  
console.log("post function");  
  
console.log("pre llamada");  
saludaPorConsola();  
console.log("post llamada");  
// El anterior código escribirá en la consola:  
// pre function  
// post function  
// pre llamada  
// Hola, soy JavaScript  
// post llamada
```

En el ejemplo anterior, el flujo de ejecución pasará por la función sin ejecutarla, pero posteriormente al llamarla por su nombre, el flujo entra dentro de la función y ejecuta ese código que está envuelto en las llaves, cuando el código de la función acaba, el flujo vuelve a donde la función se había llamado.

Parámetros de una función

Con el fin de hacer esos trozos de código más útiles y reutilizables, las funciones pueden recibir parámetros.

Los parámetros son valores que se reciben en la función y actúan como variables predeclaradas solo en el ámbito (entre llave y llave) de la función.

```
function saludar(nombre) {  
  console.log("Hola " + nombre);  
}  
  
saludar("Félix");  
// Mostrará por consola: Hola Félix  
  
saludar("Fran");  
// Mostrará por consola: Hola Fran
```

En el ejemplo anterior, se ha definido la función saludar que acepta el parámetro nombre de manera que puede haber muchos tipos de saludos, tantos como parámetros nombre reciba la función. De esta manera podemos saludar a diferentes nombres pasando como argumento de la función una cadena de texto en el momento de llamar a la función: saludar ("Félix").

Las funciones pueden tener varios parámetros, cada uno de esos parámetros se separa con «,».

```
function sumar(a, b) {  
  console.log(a + b);  
}  
  
sumar(2,3);  
// Mostrará por consola 5
```

Los parámetros de una función pueden tener un valor por defecto, de esta manera ese parámetro se convierte en opcional, ya que no es necesario pasarlo al llamar la función (hay un valor por defecto en tal caso).

```
function saludar(nombre = "Desconocido") {  
  console.log("Hola " + nombre);  
}
```

```
saludar(); // Hola Desconocido  
saludar("Víctor") // Hola Víctor
```

Retorno de una función

Ese conjunto de sentencias que es una función puede además devolver un valor, de esta manera es muy útil para abstraer cálculos y simplemente llamando a una función, se obtiene un resultado.

```
function sumar(a, b) {  
  return a + b;  
}  
  
const x = sumar(2,3);  
console.log("La suma es: " + x);  
// Mostrará por consola  
// La suma es: 5
```

En el ejemplo anterior la función sumar devuelve la suma de los dos números que entran como parámetro. Ese resultado se guarda en la constante x que posteriormente se utiliza.

Conviene saber que:

- ⊕ Una función no ejecuta más código tras encontrar la palabra return.
- ⊕ Si una función no tiene la palabra return, devolverá siempre undefined.

Ámbito de variables

El ámbito de una variable define las partes del código en la que una variable está disponible para utilizarse.

JavaScript tiene dos ámbitos de variable:

- ⊕ Ámbito de variables global.
- ⊕ Ámbito de variables local.

Ámbito global

Las variables declaradas fuera de una función se consideran variables de ámbito global. Al declararse fuera de una función estas variables están disponibles para utilizarse en cualquier parte del programa.

```
const nombre = "Gerardo";

function saludar() {
  console.log("Hola " + nombre);
}

saludar();
// Mostrará por consola
// Hola Gerardo
```

Es conveniente controlar las variables de ámbito global y utilizarlas realmente cuando se necesita que una variable esté disponible en todo el código. Las variables también se pueden modificar en cualquier parte del código y eso puede llevar a comportamientos indeseados:

```
function saludar() {
  nombre = "Juanjo";
}

let nombre = "Víctor";

saludar();
```

```
console.log("Hola " + nombre);  
// Mostrará por consola  
// Hola Juanjo
```

Ámbito local

Si una variable se declara dentro de una función, esa variable estará únicamente disponible dentro de esa función. Cada función tiene su ámbito propio llamado ámbito local.

```
const nombre = "Víctor";  
  
function saludar() {  
  const prefijo = "Hola ";  
  return prefijo + nombre;  
}  
  
console.log(saludar());  
// Hola Víctor  
  
console.log(prefijo);  
// ReferenceError: prefijo is not defined
```

En el ejemplo anterior:

- ⊕ La variable nombre es de ámbito global y por lo tanto está disponible para usar dentro de todo el código, incluido dentro de la función saludar.
- ⊕ La variable prefijo es de ámbito local ya que se ha declarado dentro de la función saludar y por lo tanto solo estará disponible dentro de dicha función. Al intentar utilizarla fuera de la función JavaScript no sabe de ella y da error.

Con la declaración de variables con let y const se puede hablar incluso de **ámbito de bloque**. Toda variable declarada de esta manera estará disponible entre las llaves {} en la que se encuentre:

```
function saludar(nombre) {  
  let prefijo = "Hola ";  
  
  if (nombre === "Félix") {  
    let sufijo = ", cuanto tiempo!";  
    console.log(prefijo + nombre + sufijo);  
  } else {  
    console.log(prefijo + nombre);  
    console.log(sufijo);  
  }  
}  
  
saludar("Félix");  
// Hola Félix, cuanto tiempo!  
  
saludar("Gerardo");  
// Hola Gerardo  
// ReferenceError: sufijo is not defined
```

Reasignación de variables

Se puede dar el caso de que se use el mismo identificador para una variable global que para una local, JavaScript está preparado para ello y utilizará primero las variables de ámbito local sobre las de ámbito global.

```
const plato = "Paella";  
  
function cocinar() {  
  const plato = "Bistec";  
  console.log("local", plato);  
}  
  
cocinar(); // local Bistec  
console.log("global", plato); // global Paella
```

Funciones anónimas

Las funciones se pueden definir como expresión, es decir, se pueden guardar en una variable. Al guardar una función en una variable **ya no es necesario asignarle nombre** por lo que se convierte en una función anónima. Para llamar a una función anónima guardada en una variable simplemente se debe escribir el nombre de esa variable seguido de ().

```
// La constante saludar guarda una función anónima
const saludar = function() {
  console.log("hola!");
}

// Llamada a la función en la variable saludar
saludar(); // hola!
```

Arrow functions

Esta nueva sintaxis para funciones se introdujo en la revisión ES6 de JavaScript, la cual hace más sencillo escribir funciones.

```
// La función sumar funciona de la misma manera en los siguientes ejemplos
const sumar = function(a, b) {
  return a + b;
}

// Equivalencia en arrow function
const sumar = (a, b) => {
  return a + b
}

// Como únicamente tiene una sentencia return, se puede quitar junto con las llaves
const sumar = (a, b) => a + b
```

Rest parameters

Uno de los problemas que se presenta en proyectos complejos es la cantidad de parámetros que finalmente puede tener una función. Utilizando un parámetro rest una función puede recibir un número indeterminado de parámetros.

Simplemente, en el último parámetro de una función se debe añadir como prefijo ..., esto hará que el resto de los parámetros pasados a la función se agrupen en un array.

```
function as(a, b, ...otros) {  
  console.log('a', a);  
  console.log('b', b);  
  console.log('otros', otros);  
}
```

```
as(1,2,3,4,5,6,7,8,9);  
// 'a' 1  
// 'b' 2  
// 'otros' [3,4,5,6,7,8,9]
```